Universidad Autónoma de Ciudad Juárez

Instituto de Ingeniería y Tecnología

Departamento de Ingeniería Eléctrica y Computación

Ingeniería de Software

"Sistema de Control de Gastos en Base al Modelo 4P+T: Mi-Monederito"

Equipo THETO

Sariña Salinas Luis Eduardo	**202245**
Esquivel García Ian Gerardo	**234646**
Beltrán Herrada Jared Gustavo	**234878**
Santillan Núñez Ana Paula	**246102**
Muñoz González Marlene Itzel	**247554**

Fundamentos de Ingeniería de Software

Maestro. Rafael Jiménez Castro

Ciudad Juárez, Chihuahua, Enero - Mayo 2026

1. Descripción general del sistema.

1.1. Nombre

Mi-Monederito.

El nombre Mi-Monederito fue elegido porque transmite de forma sencilla, cercana y fácil de recordar la idea central del sistema: ayudar a los estudiantes a administrar su dinero de manera práctica y cotidiana. La palabra *"monedero"* se asocia directamente con el lugar donde se guarda y organiza el dinero, mientras que el diminutivo *"monederito"* refleja la realidad económica de muchos estudiantes universitarios, quienes suelen manejar cantidades pequeñas de dinero y recursos limitados para cubrir sus necesidades académicas, personales y, en algunos casos, de microemprendimiento.

Además, el nombre busca proyectar una imagen amigable, accesible y cercana al público estudiantil, alejándose de términos financieros complejos que podrían generar distancia o desinterés. De esta manera, *Mi-Monederito* comunica la idea de una herramienta digital simple pero útil que acompaña al estudiante en la organización de su dinero, permitiéndole diferenciar gastos académicos, personales y de emprendimiento, reducir el estrés financiero y mejorar su estabilidad económica durante su formación universitaria.

1.2. Problema que resuelve

La falta de educación financiera práctica en el sector universitario, agravada por la mezcla de gastos personales y el microemprendimiento llevan venta de productos para solventar su carrera, genera una mala gestión (desconexión de recursos limitados y gastos para la compra de nuevos insumos) provoca que la mayoría de las veces los estudiantes no logren cubrir sus necesidades básicas semanales o incluso que opten por abandonar sus estudios.
El sistema resuelve:

- La dificultad de categorizar gastos académicos, frente a gastos personales o de emprendimiento informal.
- Estrés por deudas acumuladas o mala administración de ahorros.
- Reducir el rezago o deserción estudiantil por insolvencia económica.

1.3. Usuarios principales

- *Estudiantes foráneos:* Necesita llevar un control estricto en la renta, comida y servicios de todo tipo.
- *Estudiante con beca o apoyo:* Necesita administrar sus montos fijos para todo el semestre.
- *Estudiantes con emprendimiento informal:* Aquellos que venden algún tipo de producto dentro y fuera de la universidad para pagar estudios o libros.

1.4. Objetivo general

Desarrollar una herramienta digital móvil que le permita a los estudiantes, registrar, categorizar y analizar sus gastos, fomentando hábitos como el ahorro mediante metas personalizadas y alertas que ayuden al estudiante a controlar sus gastos, además de integrar una interfaz intuitiva reduciendo la curva de aprendizaje de la aplicación.

1.5 Objetivos Específicos

- Diseñar una interfaz móvil intuitiva que permita a los estudiantes registrar sus ingresos y gastos de manera rápida y sencilla, reduciendo la curva de aprendizaje de la aplicación.
- Implementar un sistema de categorización de gastos que permita diferenciar entre gastos académicos, personales y de emprendimiento, facilitando una mejor organización financiera.
- Desarrollar un módulo de registro y seguimiento de ingresos y egresos, que permita a los estudiantes visualizar su flujo de dinero en periodos diarios, semanales o mensuales.
- Incorporar herramientas de análisis financiero, como gráficos y resúmenes, que ayuden al estudiante a comprender sus hábitos de consumo.
- Crear un sistema de metas de ahorro personalizadas, que permita a los usuarios establecer objetivos financieros y monitorear su progreso.
- Integrar alertas y notificaciones inteligentes que adviertan al estudiante cuando se acerque a un límite de gasto o cuando esté en riesgo de exceder su presupuesto.
- Evaluar la usabilidad de la aplicación con estudiantes universitarios, con el fin de verificar que la herramienta realmente facilite la gestión de sus recursos económicos.

2. Procesos (de la aplicación)

2.1 Perfil de usuarios

Para que el sistema sea efectivo se han definido los siguientes roles con permisos específicos:

- Usuario Estudiante (Estándar): Perfil principal con acceso a registro de gustos personales y académicos.
- Administrador del Sistema: (Opcional/Técnico) Responsable del mantenimiento de las categorías globales y soporte técnico.

2.2 Casos de Uso Principales

- Registro de Gasto/Ingreso: El usuario captura una transacción financiera.
- Configuración de Metas de Ahorro: Definición de un objetivo (ejemplo: Inscripción) y el monto a separar.
- Consulta de reportes: Visualización de estadísticas de consumo semanal/mensual.

2.3 Descripción de los Procesos Principales

2.3.1. Proceso 1: Registro y categorización Inteligente

Nombre del proceso: Registro de Transacción Hibrida.

Descripción paso a paso:

1. El usuario selecciona el tipo de movimiento (Ingreso o Egreso).
2. El sistema solicita el monto y la Categoría Maestra (Personal, Académica o Emprendimiento).
3. El usuario selecciona una subcategoría (ejemplo Copias, Renta, Insumos de venta).
4. El sistema valida si el gasto afecta una Meta de Ahorro activa o si sobrepasa un límite de presupuesto.
5. Se guarda el registro en la base de datos y se actualiza el saldo en tiempo real en el Dashboard.

2.3.2. Proceso 2: Monitoreo de Alertas de Solvencia

Nombre del proceso: Sistema de Alerta preventiva de Deserción.

Descripción:

1. El sistema analiza el promedio de gasto semanal del usuario frente a su saldo actual.
2. Si el saldo proyectado es insuficiente para cubrir los gastos fijos del mes, se recibirá una notificación.
3. El sistema propone un Ajuste de Gasto Hormiga sugiriendo categorías donde el usuario puede recortar egresos para priorizar la universidad.

2.4. Proceso de desarrollo

2.4.1. Metodología de desarrollo elegida

Scrum. La metodología Scrum es un marco de trabajo ágil utilizado para gestionar y desarrollar proyectos, especialmente en el desarrollo de software, se basa en la colaboración del equipo, la entrega continua de resultados y la mejora constante a través de ciclos de trabajo cortos llamados Sprints, que normalmente duran entre una y cuatro semanas.

Durante cada Sprint, el equipo selecciona un conjunto de tareas priorizadas del Product Backlog para desarrollarlas y entregar un incremento funcional del producto. Este proceso permite adaptar el proyecto a cambios o nuevas necesidades, ya que al finalizar cada ciclo se revisan los resultados obtenidos y se ajustan los siguientes pasos.

2.4.2. Justificación

Se eligió la metodología Scrum para el desarrollo del proyecto Mi-Monederito debido a que es una metodología de trabajo ágil que permite desarrollar aplicaciones de forma incremental, flexible y centrada en el usuario. Dado que el proyecto consiste en una herramienta móvil dirigida a estudiantes universitarios, es importante poder adaptar el desarrollo conforme se obtenga retroalimentación de los usuarios, algo que Scrum facilita mediante ciclos de trabajo cortos llamados Sprints.

Esta metodología permite dividir el desarrollo del sistema en pequeñas funcionalidades, como el registro de gastos, la categorización, las metas de ahorro y el sistema de alertas, las cuales pueden implementarse, probarse y mejorarse progresivamente. De esta manera, el equipo puede evaluar continuamente el funcionamiento de la aplicación y realizar ajustes oportunos, asegurando que la herramienta sea realmente útil e intuitiva para los estudiantes.

Además, para el desarrollo del proyecto estaremos trabajando con jira, por lo que Scrum es una de las metodologías que mejor funciona con esta herramienta, contemplando que fomenta la colaboración constante entre los miembros del equipo, la organización del trabajo y la revisión periódica de los avances, lo que ayuda a mejorar la calidad del producto final. Esto es especialmente importante en proyectos académicos o de desarrollo tecnológico, donde el tiempo y los recursos suelen ser limitados.

2.5. Actividades principales

En particular la metodología elegida, no cuenta con fases específicas de desarrollo, pero tiene actividades claves que se realizan en cada sprint que hemos asociado con las siguientes fases, por lo que idealmente trabajamos en un ciclo iterativo de estas fases durante cada sprint.

2.5.1. Análisis / Product Backlog

En esta fase se identificará con claridad el problema que busca resolver Mi Monederito, así como las necesidades reales de los usuarios principales que ya han sido mencionados anteriormente y también se recopilarán los requerimientos funcionales y no funcionales del sistema, tales como registro de ingresos, gastos, establecimiento de metas de ahorro, etc.

Al igual que en esta etapa se definirán los objetivos del sistema, el alcance inicial del producto y las prioridades principales, evitando funciones innecesarias. Así que el resultado esperado de esta fase será una lista clara de necesidades, problemas, objetivos y funcionalidades esenciales.

Dentro del enfoque de Scrum, esta etapa se relaciona con la creación y refinamiento del Product Backlog, donde se registran todas las funcionalidades y necesidades del sistema en forma de historias de usuario. En esta etapa se priorizan los requerimientos más importantes para el usuario y se define el alcance inicial del producto, evitando incluir funciones innecesarias.

2.5.2. Diseño / Planificación

En esta fase se planteará cómo se verá y cómo funcionará el sistema. Primero se diseñará como será en general la aplicación móvil lo que definirá las pantallas principales, la navegación entre ellas y la organización de la información. Se propondrá una interfaz intuitiva para que los usuarios puedan entender en qué está gastando su dinero.

También se modelará la base de datos o estructura de almacenamiento para guardar ingresos, egresos, categorías, metas y perfiles de usuario. Al final de esta fase se deberá de contar con bocetos, diagramas y una base clara para comenzar la construcción del sistema.

En relación con Scrum, esta fase se desarrolla como parte de la planificación de los Sprints, donde el equipo analiza las historias de usuario seleccionadas y define la forma en que serán implementadas. Aquí se elaboran bocetos, diagramas de navegación, prototipos de interfaz y el diseño de la base de datos o estructura de almacenamiento.

2.5.3. Desarrollo

En esta etapa se realizará la construcción del sistema. Se programarán las funciones principales de la aplicación ya sea como el registro de gastos, ingresos, la consulta de reportes, la administración de metas de ahorro y la emisión de algunos recordatorios.

De igual forma, se implementará el almacenamiento de datos. Si el proyecto lo necesita, también se integrarán componentes como autenticación de usuarios o gráficas de análisis en lo que gastan los usuarios, lo que hace que el objetivo de esta fase sea convertir las ideas y diseños previos en un producto funcional.

En el marco de Scrum, el desarrollo se lleva a cabo dentro de los Sprints, donde el equipo implementa las funcionalidades priorizadas del *Product Backlog*. Durante cada Sprint se construyen incrementos funcionales del sistema que pueden ser revisados y evaluados por el equipo.

2.5.4.Pruebas

En esta fase se verificará que la aplicación funcione bien y que realmente ayude al usuario a administrar su dinero. Se probará que cada función cumpla con lo esperado.

También se realizarán pruebas de usabilidad para comprobar si la interfaz es fácil de entender, al igual que se revisarán posibles fallos técnicos, errores en cálculos o problemas de navegación. Esta etapa nos permitirá detectar y corregir errores de tal manera que se mejore la calidad y confiabilidad del sistema.

Dentro de Scrum, las pruebas forman parte del proceso continuo dentro de cada Sprint, ya que cada incremento del producto debe ser evaluado antes de considerarse terminado. En esta etapa se realizarán pruebas funcionales, pruebas de usabilidad y revisiones técnicas para detectar errores en cálculos, fallos del sistema o problemas en la navegación de la aplicación.

2.5.5. Entrega / Review / Retrospective

En la fase de entrega se presentará la versión final del sistema ya funcional. Se dejará preparada la aplicación para su uso, incluyendo sus funciones principales, su interfaz terminada y la información necesaria para que pueda ser comprendida por los usuarios y el evaluador del proyecto. También se podrán incluir manuales breves de uso, explicación de las pantallas principales y demostraciones del funcionamiento general.

Además, en esta etapa se documentaran las características finales del sistema. Así que no solo consiste en mostrar que el sistema funciona, sino en demostrar que responde al problema planteado y que ofrece una solución útil para los estudiantes o usuarios.

En el contexto de Scrum, esta fase se relaciona con la Sprint Review, donde se presenta el incremento desarrollado y se demuestra el funcionamiento del sistema. Además, se documentarán las características finales de la aplicación y se podrán incluir manuales breves de uso, explicaciones de las pantallas principales y demostraciones del funcionamiento general.

2.6. Flujo de trabajo

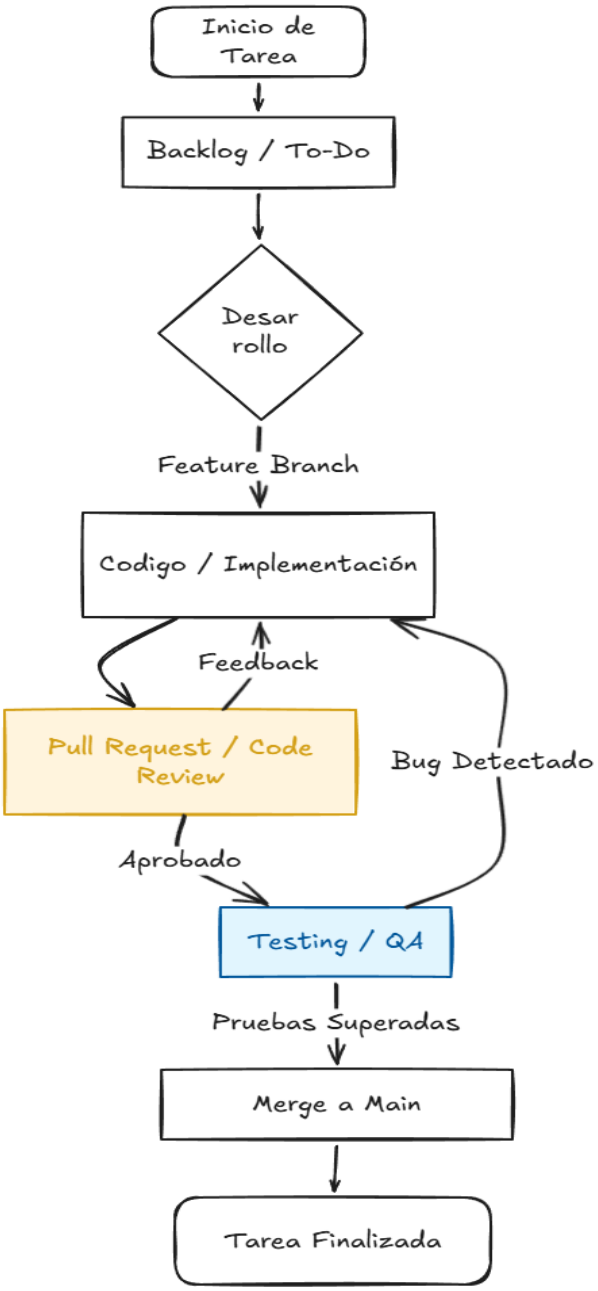

Figura 1. Diagrama de metodología de Autoría propia

2.7. Control de calidad

2.7.1. ¿Cómo evitarán errores?

Utilizaremos el método Fail Fast donde se trabaja a base de Sprints, donde cada uno es un incremento funcional del producto que se revisa y valida antes de mandarlo a main.

- Aislamiento de cambios: Cada nueva funcionalidad se trabajara en una branch independiente para así evitar afectar la versión estable de la aplicación.
- Validación cruzada: Ningún código se implementará a la main branch sin revisión de algún otro miembro de equipo para poder detectar errores que el autor original pudo no haber visto.
- Pruebas: antes de la entrega final de la feature se realizarán pruebas que simulen un uso real para identificar errores o problemas de rendimiento.

2.7.2. ¿Qué prácticas utilizaran?

- *Revisiones de Código:* Usaremos Pull Requests en github. Se verificará que el código esté documentado y siga estándares definidos.
- *Smoke testing:* Al final de cada fase, se realizan pruebas rápidas de las funciones vitales para asegurar que las funciones básicas que no se hayan roto con la implementación nueva que se quiere hacer.
- *Gestión de incidencias en tablero ágil:* Utilizaremos Jira para etiquetar y priorizar errores encontrados. Los Bugs tendrán alta prioridad sobre agregar nuevas funciones.
- *Criterios de aceptación:* Cada tarea se considera completa solo si cumple con los requisitos y ha pasado prueba de funcionalidad sin errores críticos.
-

3. Proyecto

3.1. Definición del proyecto

El proyecto Mi-Monederito consiste en el desarrollo de una aplicación móvil orientada a estudiantes universitarios que permita registrar, organizar y analizar sus ingresos y gastos de manera sencilla. La aplicación busca apoyar la educación financiera práctica de los estudiantes, facilitando la diferenciación entre gastos personales, académicos y relacionados con microemprendimientos.

El sistema permitirá a los usuarios llevar un control básico de su dinero, establecer metas de ahorro y recibir alertas o recordatorios relacionados con sus hábitos de gasto. De esta forma, la aplicación pretende ayudar a mejorar la administración de los recursos económicos de los estudiantes y fomentar hábitos financieros responsables.

3.1.1. Alcance

El alcance del proyecto comprende el diseño, desarrollo y presentación de una aplicación móvil funcional que permita a los estudiantes registrar y gestionar sus finanzas personales de forma sencilla.
El sistema incluirá funcionalidades como el registro de ingresos y gastos, la clasificación de gastos por categorías (personales, académicos o de emprendimiento), la visualización de reportes básicos de gastos, el establecimiento de metas de ahorro y la generación de alertas o recordatorios relacionados con el control financiero.

Además, el proyecto contempla el diseño de una interfaz intuitiva, la implementación de una estructura de almacenamiento de datos, y la realización de pruebas de funcionamiento y usabilidad para asegurar que la aplicación sea comprensible y útil para los estudiantes.
Sin embargo, el proyecto no incluirá integración con bancos, sistemas de pago en línea ni manejo de transacciones financieras reales, ya que su objetivo principal es servir como una herramienta de control y organización de gastos personales, sin mencionar que está fuera de las restricciones principales de un proyecto escolar.

3.1.2. Entregables

Como resultado del desarrollo del proyecto se entregarán los siguientes productos:

- Aplicación móvil funcional de Mi-Monederito con las características principales del sistema.
- Diseño de interfaz de usuario, incluyendo bocetos o prototipos de las pantallas principales.
- Modelo de base de datos o estructura de almacenamiento utilizada para el registro de información financiera.
- Documentación del proyecto, donde se describa el análisis, diseño, desarrollo y pruebas del sistema.
- Manual básico de usuario, que explique el funcionamiento general de la aplicación.
- Demostración del sistema, donde se muestre el funcionamiento de las principales funciones de la aplicación.

Estos entregables permitirán evidenciar el proceso de desarrollo del sistema y demostrar que la solución propuesta responde al problema planteado.

3.2. Planificación básica (en Jira)

3.2.1. Cronograma

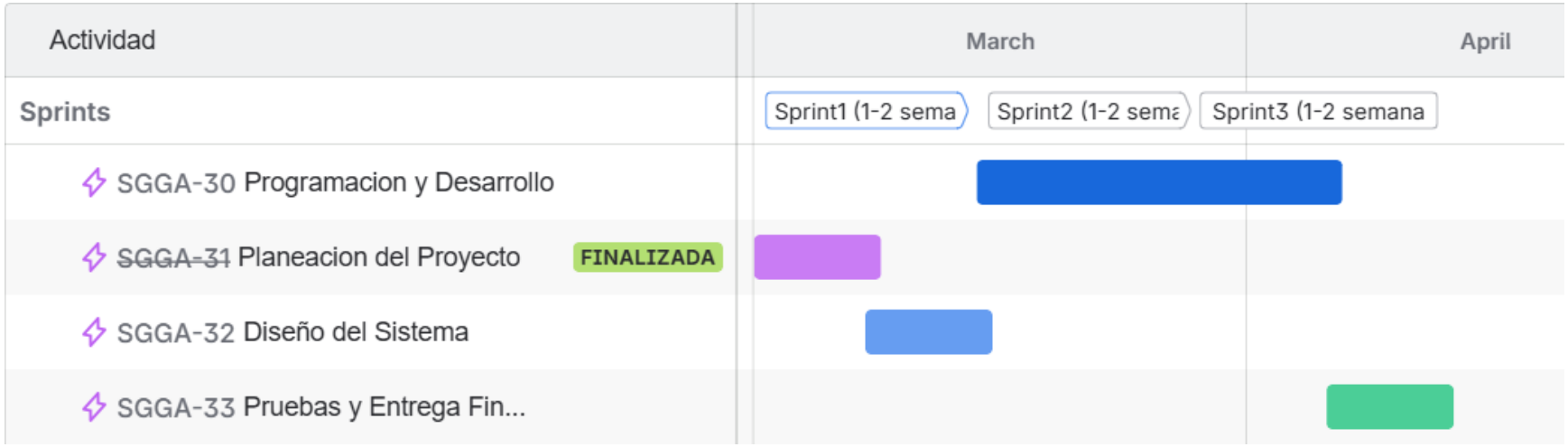

Figura 2. Cronograma de Proyecto Fragmento de planificación en Jira de Autoría propia

3.2.2. Lista de Actividades - Backlog

Jira

Proyecto: Sistema Gestión Gastos Académico
Ordenado por: Creada ascendente
1–27 de **27** como en: **08/mar/26 8:57 PM**

T	Clave	Resumen	Persona asignada	Informador	Pr	Estado	Resolución	Creada	Actualizada	Fecha de vencimiento
☑	SGGA-1	Definir el Proyecto	*sin asignar*	JARED GUSTAVO BELTRAN HERRADA	⌃⌃	FINALIZADA	Listo	07/mar/26	08/mar/26	08/mar/26
☑	SGGA-2	Creación de espacio en Jira	ANA PAULA SANTILLAN NUÑEZ	ANA PAULA SANTILLAN NUÑEZ	=	FINALIZADA	Listo	07/mar/26	08/mar/26	08/mar/26
☑	SGGA-4	Definición del Product Backlog	ANA PAULA SANTILLAN NUÑEZ	ANA PAULA SANTILLAN NUÑEZ	=	EN CURSO	*Sin resolver*	08/mar/26	08/mar/26	08/mar/26
☑	SGGA-5	Análisis de requerimientos del sistema	JARED GUSTAVO BELTRAN HERRADA	ANA PAULA SANTILLAN NUÑEZ	⌃⌃	TAREAS POR HACER	*Sin resolver*	08/mar/26	08/mar/26	15/mar/26
☑	SGGA-6	Diseño de la estructura general de la aplicación	Marlene Itzel Muñoz Gonzalez	ANA PAULA SANTILLAN NUÑEZ	⌃	TAREAS POR HACER	*Sin resolver*	08/mar/26	08/mar/26	15/mar/26
☑	SGGA-7	Diseño de la interfaz básica	*sin asignar*	ANA PAULA SANTILLAN NUÑEZ	⌄	TAREAS POR HACER	*Sin resolver*	08/mar/26	08/mar/26	15/mar/26
☑	SGGA-9	Desarrollo del módulo de registro de ingresos	luis Eduardo Sariñana salinas	ANA PAULA SANTILLAN NUÑEZ	=	TAREAS POR HACER	*Sin resolver*	08/mar/26	08/mar/26	22/mar/26
☑	SGGA-10	Desarrollo del módulo de registro de gastos	Ian Esquivel	ANA PAULA SANTILLAN NUÑEZ	=	TAREAS POR HACER	*Sin resolver*	08/mar/26	08/mar/26	22/mar/26
☑	SGGA-11	Implementación de categorías de gastos	JARED GUSTAVO BELTRAN HERRADA	ANA PAULA SANTILLAN NUÑEZ	=	TAREAS POR HACER	*Sin resolver*	08/mar/26	08/mar/26	29/mar/26
☑	SGGA-12	Desarrollo de la estructura de almacenamiento de datos	ANA PAULA SANTILLAN NUÑEZ	ANA PAULA SANTILLAN NUÑEZ	=	TAREAS POR HACER	*Sin resolver*	08/mar/26	08/mar/26	29/mar/26

Figura 3. Fragmento del Backlog del proyecto en Jira de Autoría propia

SGGA-17	Preparación de la versión final del sistema	Ian Esquivel	ANA PAULA SANTILLAN NUÑEZ		TAREAS POR HACER	*Sin resolver*	08/mar/26	08/mar/26	12/abr/26
SGGA-18	Documentacion: Seccion 1	JARED GUSTAVO BELTRAN HERRADA	ANA PAULA SANTILLAN NUÑEZ		FINALIZADA	Listo	08/mar/26	08/mar/26	08/mar/26
SGGA-19	Documentacion:Seccion 2	luis Eduardo Sariñana salinas	ANA PAULA SANTILLAN NUÑEZ		FINALIZADA	Listo	08/mar/26	08/mar/26	08/mar/26
SGGA-20	Documentacion:Seccion 3	ANA PAULA SANTILLAN NUÑEZ	ANA PAULA SANTILLAN NUÑEZ		EN CURSO	*Sin resolver*	08/mar/26	08/mar/26	08/mar/26
SGGA-21	Documentacion:Seccion 4	luis Eduardo Sariñana salinas	ANA PAULA SANTILLAN NUÑEZ		TAREAS POR HACER	*Sin resolver*	08/mar/26	08/mar/26	15/mar/26
SGGA-22	Documentacion:Seccion 5	Ian Esquivel	ANA PAULA SANTILLAN NUÑEZ		TAREAS POR HACER	*Sin resolver*	08/mar/26	08/mar/26	15/mar/26
SGGA-23	Documentacion:Seccion 6	Marlene Itzel Muñoz Gonzalez	ANA PAULA SANTILLAN NUÑEZ		TAREAS POR HACER	*Sin resolver*	08/mar/26	08/mar/26	15/mar/26
SGGA-24	SGGA-19 Procesos	luis Eduardo Sariñana salinas	ANA PAULA SANTILLAN NUÑEZ		TAREAS POR HACER	*Sin resolver*	08/mar/26	08/mar/26	
SGGA-25	SGGA-19 Actividades	Marlene Itzel Muñoz Gonzalez	ANA PAULA SANTILLAN NUÑEZ		TAREAS POR HACER	*Sin resolver*	08/mar/26	08/mar/26	
SGGA-26	SGGA-19 Flujo	Ian Esquivel	ANA PAULA SANTILLAN NUÑEZ		TAREAS POR HACER	*Sin resolver*	08/mar/26	08/mar/26	
SGGA-27	SGGA-19 Control de calidad	Ian Esquivel	ANA PAULA SANTILLAN NUÑEZ		TAREAS POR HACER	*Sin resolver*	08/mar/26	08/mar/26	
SGGA-28	SGGA-20 Definicion y Planificacion	ANA PAULA SANTILLAN NUÑEZ	ANA PAULA SANTILLAN NUÑEZ		FINALIZADA	Listo	08/mar/26	08/mar/26	
SGGA-29	SGGA-20 Recursos, riesgos y Metricas	Ian Esquivel	ANA PAULA SANTILLAN NUÑEZ		EN CURSO	*Sin resolver*	08/mar/26	08/mar/26	

Figura 4. Fragmento de Backlog del proyecto en Jira de Autoría propia

3.2.3. Sprints

3.2.3.1. Sprint 1

Sprint1 (1-2 semanas) 1 mar – 15 mar (7 actividades) | 7 | 12 | 3 | Completar sprint

- Definir el proyecto - Entrega y desarrollo de las primeras tres partes del proyecto 1. Descripción general del sistema 2. Procesos 3. Proyecto - Implementación de Jira y Github

~~SGGA-1~~	Definir el Proyecto	FINALIZADA	8 mar	3	
SGGA-5	Análisis de requerimientos del sistema	EN CURSO	15 mar	5	JH
SGGA-6	Diseño de la estructura general de la aplicación	TAREAS POR HACER	15 mar	5	MG
SGGA-7	Diseño de la interfaz básica	EN CURSO	15 mar	3	AN
SGGA-21	Documentacion:Seccion 4	EN CURSO	15 mar	2	
SGGA-22	Documentacion:Seccion 5 + Epic	TAREAS POR HACER	15 mar	2	IE
SGGA-23	Documentacion:Seccion 6	EN CURSO	15 mar	2	MG

Figura 5. Imagen Sprint 1 del Proyecto en Jira de Autoría propia

3.2.3.2. Sprint 2

Sprint2 (1-2 semanas) 15 mar – 29 mar (5 actividades) | 31 | 0 | 0 | Iniciar sprint

Implementar el módulo básico de reportes para que los administradores puedan ver ventas.

SGGA-9	Desarrollo del módulo de registro de ingresos	TAREAS POR HACER	22 mar	7	
SGGA-10	Desarrollo del módulo de registro de gastos	TAREAS POR HACER	22 mar	6	IE
SGGA-11	Implementación de categorías de gastos	TAREAS POR HACER	29 mar	8	JH
SGGA-12	Desarrollo de la estructura de almacenamiento de datos	TAREAS POR HACER	29 mar	4	AN
SGGA-13	Implementación de metas de ahorro	TAREAS POR HACER	5 abr	6	MG

Figura 6. Imagen Sprint 2 del Proyecto en Jira de Autoría propia

3.2.3.3. Sprint 3

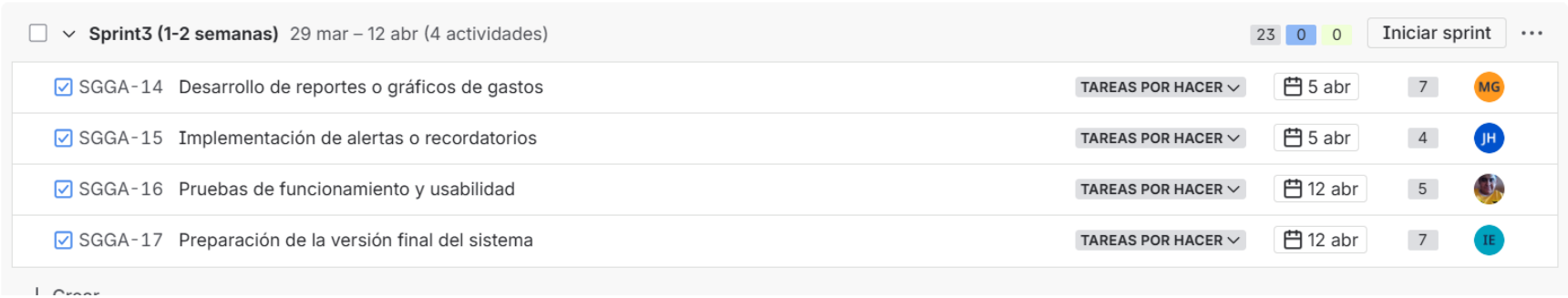

Figura 7. Imagen Sprint 3 del Proyecto en Jira de Autoría propia

3.3. Recursos

3.3.1. Humanos

- Analistas y Desarrolladores: Esos vamos a ser todos en mayor o menor medida en el equipo para realizar el proyecto.
- Clientes: Pueden ser amigos que prueben la aplicación sin nosotros decirle cómo funciona al 100% más que una breve descripción para saber si funciona como queremos que funcione en un público que no lo desarrolla.

3.3.2. Técnicos

- Stack:
    - Gestión: Jira para el seguimiento de tareas y cronogramas
    - Github: Para el control del código y los pull requests.
- Hardware: De cualquier tipo para que funcione como queremos como pueden ser laptops para programar o crear el servidor local o los celulares para probar la aplicación.

3.4. Riesgos identificados

Riesgo identificado	Categoría	Plan de mitigación
Pérdida de avance por fallos	Tecnología	Vamos a utilizar GitHub para que siempre haya un lugar donde tener guardada la aplicación completa.
Retraso en el flujo de trabajo	Proceso	Estar revisando cada cierto tiempo qué se está avanzando y qué no, para saber en qué debemos centrarnos.
Falta de documentación	Proceso	Antes de aceptar cualquier código en la *main branch*, se debe revisar que esté bien documentado para que, si alguien quiere cambiar algo, sepa qué hace cada parte del código.
Pruebas	Proceso	Debemos realizar pruebas para asegurar que todo funcione como se espera antes de hacer *pull* a la *main branch*, para que nada crítico se rompa.
Errores de cálculos y balance	Tecnología	Se realizarán pruebas para mantener bien los balances y que la aplicación arroje correctamente los resultados.

Tabla 1. Registro detallado de los posibles riesgos del proyecto de Autoría Propia

3.5. Métricas del proyecto

3.5.1. Tiempo

Tenemos la mayor parte del semestre para hacer el proyecto y poder avanzar en base a eso no debería de ser problema, así que toda la parte de desarrollo, pruebas y documentación no debería de ser un problema.

3.5.2. Costo estimado

El desarrollo no tiene un costo al utilizar herramientas de uso gratuito y nosotros ya tener el hardware de antes de siquiera pensar en hacer este proyecto, lo único que gastaremos es nuestro tiempo lo cual ya era esperado.

3.5.3. Indicadores del avance

Se pueden considerar los siguientes puntos para medir:

- Porcentaje de funcionalidades implementadas respecto al plan inicial.
- Cumplimiento del cronograma establecido.

4. Personas

4.1. Roles

En esta etapa tomamos la base de nuestra metodología considerando que presenta tres roles principales

4.1.1. Product Owner

Es la persona responsable de definir qué debe desarrollarse en el producto y de priorizar las tareas en el Product Backlog, es importante porque asegura que el equipo trabaje en las funciones que realmente aportan valor al cliente o al negocio.

Contemplando esas características principales identificamos que en nuestro equipo había una persona que podía desenvolverse confiadamente en este rol, por lo que decidimos darle la formalidad del rol a Jared Beltrán, además de que sabemos que el se puede desenvolver en este rol, también consideramos que su participación desde esta perspectiva ayudará al resto del grupo en el proyecto.

4.1.2. Scrum Master

Es el facilitador del equipo, se encarga de que el equipo siga correctamente la metodología Scrum y de eliminar obstáculos durante el Sprint, su función comprende ayudar al equipo a trabajar de forma eficiente y a mejorar continuamente el proceso.

Desde nuestro equipo tomamos la decisión de que Ana Santillán desempeñe este rol en la ejecución del proyecto, creemos que es un rol que exige habilidades que podemos encontrar en este integrante del equipo.

4.1.3. Development Team

Es el grupo de profesionales que diseña, desarrolla y prueba el producto durante cada sprint, son parte importante porque son quienes convierten las ideas y requisitos en un producto funcional.

Aunque en esta metodología no define otros roles para el equipo de desarrollo, particularmente nosotros definimos algunos, que nos ayudan a determinar cuales son las responsabilidades de cada uno.

4.1.3.1. Responsable de Procesos:

Es el miembro del equipo encargado de diseñar, organizar y supervisar la lógica de funcionamiento de los procesos internos del sistema, en nuestro proyecto desempeñado por Luis Sariñana

4.1.3.2. Responsable de Interfaz de Usuario:

Encargado del diseño y desarrollo de la interfaz visual del sistema, asegurando que la aplicación sea intuitiva, clara y fácil de usar para los usuarios, esta parte del proceso será desempeñada por Ian Esquivel.

4.1.3.3. Responsable de Base de Datos y Pruebas:

Encargado de la estructuración, administración y optimización de la base de datos del sistema, garantizando la correcta gestión y almacenamiento de la información, además es el miembro del equipo encargado de verificar que el sistema funcione correctamente y cumpla con los requisitos establecidos, esto implica realizar pruebas sobre las distintas funcionalidades del sistema para detectar errores, fallos o inconsistencias antes de que el producto sea entregado, rol desempeñado por Marlene Muñoz.

4.1.4. Resumen de roles

Roles		**Integrante**
Product Owner		Jared Beltran
Scrum Master		Ana Santillan
Development Team	Responsable de Procesos	Luis Sariñana
	Responsable de Interfaz de Usuario	Ian Esquivel
	Responsable de Bases de Datos y Pruebas	Marlene Muñoz

Tabla 2. Resumen de roles del equipo conforme a la metodología de Autoría propia

4.2. Responsabilidades

Roles		Responsabilidades
Product Owner		Definir la visión del producto, gestionar y priorizar el Product Backlog, comunicar requisitos al equipo, validar las funcionalidades desarrolladas y mantener comunicación con los interesados del proyecto.
Scrum Master		Facilitar las reuniones del equipo, eliminar obstáculos que afecten el desarrollo, guiar al equipo en las prácticas de Scrum, fomentar la comunicación y apoyar la mejora continua del proceso.
Development Team	*Responsable de Procesos*	Diseño de la lógica de negocio, validación de flujos de datos y estructuración de los procesos internos de cálculo.
	Responsable de Interfaz de Usuario	Diseño de las pantallas del sistema, implementación de componentes visuales y mejora de la experiencia de usuario en las funcionalidades principales de la plataforma.
	Responsable de Base de Datos y Pruebas	Diseño del esquema de la base de datos, gestión de consultas y mantenimiento de la integridad de los datos utilizados por el sistema, ejecución de pruebas funcionales del sistema, detección y reporte de errores, así como validación del correcto funcionamiento de las funcionalidades desarrolladas.

Tabla 3. Descripción de responsabilidades de cada rol de Autoría propia

4.3. Estrategia de comunicación

Para asegurar la agilidad que tiene la metodología SCRUM, el equipo utilizará un esquema de comunicación híbrido:

- *Gestión de tareas y avance:* Se utilizará Jira para dar seguimiento a tareas, cronogramas e indicadores de avance.
- *Comunicación técnica y código:* Se usará GitHub para la revisión de código y retroalimentación técnica.
- *Comunicación asincrónica*: Se llevarán a cabo reuniones periódicas (acordes a los ciclos establecidos en los Sprints) para la revisión de avances y coordinación.
- *Comunicación inmediata:* Se utilizará WhatsApp para la resolución de dudas rápidas y avisos urgentes que no requieran ser documentados.

4.4. Reglas internas del equipo

El equipo seguirá las siguientes reglas para asegurar la calidad y el cumplimiento de objetivos del proyecto:
Aislamiento de cambios: Todo nuevo desarrollo deberá trabajarse en una rama independiente para no comprometer la versión estable.
Validación cruzada: Ningún integrante del equipo puede subir código a la rama principal (versión estable) si no ha sido revisada y aprobada anteriormente por algún otro miembro del equipo, esto con el fin de detectar errores omitidos por el autor original.
Criterios de aceptación: Una tarea sólo será marcada como finalizada en "Jira" si cumple con los requisitos anteriormente definidos.
Prioridad de bugs: La corrección de bugs o errores tendrá una prioridad absoluta, esto con el fin de asegurar la mayor calidad del producto.

4.5. Reglas para el manejo de conflictos

Para resolver cualquier conflicto de índole técnico o de procesos, se seguirán los siguientes pasos:

- Referencia al product backlog: Ante dudas sobre que funcionalidad se debe desarrollar, siempre se tendrá como referencia la priorización establecida en Product Backlog.
- Consenso de retrospectivas: Se hará uso de sesiones de Retrospective al final de
- cada sprint, con el fin de detallar estragos en el flujo de trabajo.
- Escalamiento preventivo: Se mantendrá énfasis en el flujo de trabajo para identificar retrasos y centrarnos en las prioridades, esto con el fin de evitar conflictos de entrega.

5. Producto

5.1. Funcionalidades principales

- *Dashboard Mensual de control:*
  Pantalla principal que muestra de forma gráfica la distribución del dinero entre viviendo, Servicios de Escuela y Otros, permitiendo un balance inmediato entre ingresos y Egresos
- *Gestión de Guardaditos:*
  Función para separar montos de dinero que no pueden ser utilizados hasta el final del mes, ayudando a evitar el gasto impulsivo.
- *Módulo de planes de Ahorro:*
  Permite configurar metas específicas (ejemplo: $2000 en 10 meses) calculando automáticamente el resguardo mensual necesario.
- *Control de Préstamos:*

Herramienta para organizar y dar seguimiento a los pagos de deudas o préstamos realizados, centralizando los compromisos financieros.

- *Límites de Gastos por categoría:*
  Permite establecer topes máximos de gasto (ejemplo: $200 para ocio) con indicadores visuales de proximidad al límite.
- *Historial detallado*:
  Listado cronológico de entradas y salidas para auditoría personal del usuario.

5.2. Requisitos Básicos

Para que el sistema funcione correctamente según lo planeado por equipo de desarrollo, se deben cumplir los siguientes requisitos mínimos:

5.2.1. Requisitos Funcionales (RF)

- *RF-01 Registro y Acceso:*
  El sistema debe permitir la creación de cuentas (Nombre, Correo, Contraseña) y validación de identidad para proteger los datos financieros.
- *RF-02 Gestión de Transacciones:*
  El usuario debe poder sumar ingresos y restar egresos de forma manual, vinculándolos a una categoría.
- *RF-03 Calculo automático de Ahorros:*
  El sistema debe calcular la cuota mensual necesaria cuando el usuario ingresa un monto total y un plazo en meses.
- *RF-04 Visualización de Estadísticas*:
  El sistema debe transformar los datos numéricos en gráficos de distribución porcentual de forma automática.

5.2.2. Requisitos No Funcionales (RNF)

- *RNF-01 Persistencia de Datos:*
  La información debe almacenarse de forma estructurada para que el historial sea accesible en cualquier momento (Responsabilidad del área de Base de Datos).
- *RNF-02 Intuitividad de Interfaz:*
  El diseño debe seguir el esquema de navegación de 3 a 4 clics máximo para funciones críticas, manteniendo la estética presentada en el prototipo
- *RNF-03 Rendimiento:*
  El dashboard debe cargar y actualizar los saldos en un tiempo menor a 2 segundos tras realizar un registro.
- *RNF-04 Seguridad de Perfil:*

Posibilidad de cerrar sesión (Logout) para evitar que terceros accedan a la información financiera en el mismo dispositivo.

5.3. Características del software

El sistema Mi-Monederito debe tener algunas características importantes, las cuales deben hacer que la aplicación sea confiable, fácil de usar y útil para los estudiantes.

5.3.1. Usabilidad

La usabilidad es muy importante en el sistema. Los estudiantes son los que van a usar la aplicación para registrar sus gastos. Por eso, el diseño de la aplicación debe hacer que sea fácil moverse por la aplicación sobre todo en:

- Registro de ingresos.
- Registro de gastos.
- Visualización de reportes.
- Metas de Ahorro.
- Alertas financieras.

También es importante que la aplicación minimice la curva de aprendizaje sin necesidad de usar instrucciones complejas.

5.3.2. Rendimiento y rapidez

El sistema debe ofrecer respuestas rápidas al usuario cuando este registre una transacción, consulte reportes o revise su saldo disponible.
Las operaciones principales del sistema incluyen:

- Guardar una transacción financiera.
- Actualizar el saldo del usuario.
- Calcular reportes semanales o mensuales.

Un buen rendimiento aumenta la probabilidad de que el sistema sea más utilizado.

5.3.3. Seguridad

Aunque el sistema no realiza transacciones bancarias reales, la información sigue siendo sensible. Por esta razón, el sistema debe implementar seguridad.
Entre los aspectos que se debe considerar la aplicación se encuentran:

- Protección de los datos personales del usuario.
- Prevención de pérdida de información.
- Manejo seguro del almacenamiento de datos.

5.3.4. Confiabilidad

El sistema debe garantizar que la información registrada sea almacenada correctamente y que los cálculos financieros sean precisos.
Por ejemplo, cuando un usuario registra un gasto, el sistema debe actualizar correctamente:

- El saldo actual.
- El balance de categorías.
- El progreso de metas de ahorro.

Si el sistema presenta errores en estos cálculos, el usuario podría perder confianza.

5.3.5. Disponibilidad

La aplicación debe estar disponible para los usuarios en todo momento, así pueden registrar gastos cuando lo necesiten.

5.3.6. Escalabilidad

El sistema debe diseñarse para que sea fácil agregar nuevas funciones más adelante. Por ejemplo, en versiones futuras podrían añadirse características como:

- Sincronización con servicios financieros.
- Exportación de reports.
- Análisis más avanzados de hábitos de gasto.
- Integración con servicios en la nube.

Por lo que diseñar el sistema con una arquitectura escalable facilitará los cambios sin necesidad de rediseñar todo.

5.4. Criterios de calidad

Para garantizar que el sistema Mi-Monederito cumpla con los objetivos del proyecto, se establecen criterios de calidad que permitan evaluar el funcionamiento.

5.4.1. Correcto funcionamiento de las funcionalidades

El sistema debe implementar correctamente las funciones principales definidas en el proyecto, entre ellas:

- Registro de ingresos.
- Registro de gastos.
- Clasificación por categorías.
- Reportes de consumo.
- Metas de Ahorro.
- Sistema de alertas.

5.4.2. Precisión de los cálculos

El sistema debe realizar correctamente los cálculos relacionados con:

- Balance financiero.
- Total de gastos por categoría.
- Progreso de metas de Ahorro.
- Análisis de consumo semanal o mensual.

La precisión de estos cálculos es para que los usuarios puedan confiar en la información que proporcionada.

5.4.3. Experiencia de usuario

Para evaluar esto se pueden realizar pruebas de usabilidad con estudiantes, verificando que puedan completar tareas como:

- Registrar un gasto.
- Consultar un reporte.
- Crear una meta de Ahorro.

Si los usuarios encuentran dificultades para realizar estas tareas, el diseño del sistema debe ser mejorado.

5.4.4. Mantenibilidad del sistema

El sistema debe ser desarrollado con código organizado y documentado para facilitar futuras modificaciones o mejoras.
El uso de herramientas como GitHub para llevar un control de las diferentes versiones y revisiones del código nos ayuda a mantener un orden. Además, GitHub nos permite trabajar juntos de forma colaborativa.

5.4.5. Pruebas del sistema

Antes de considerar que una funcionalidad está terminada, esta debe pasar diferentes tipos de pruebas:

- Pruebas funcionales para verificar que las funciones estén haciendo lo deseado.
- Pruebas de usabilidad para evaluar la interacción del usuario con Mi Monederito.
- Pruebas de rendimiento para verificar la rapidez del sistema.

Estas pruebas ayudan a detectar errores.

5.5. ¿Qué haría que el producto fracase?

El fracaso del sistema puede ocurrir si ciertos riesgos no son identificados o gestionados adecuadamente.

5.5.1. Falta de facilidad de uso

Si una aplicación es complicada o lleva mucho tiempo para registrar un gasto, los usuarios no la utilizarán. Una mala experiencia de usuario puede hacer que una aplicación fracase.

5.5.2. Problemas técnicos o fallos del sistema

Errores en el código, fallos en la base de datos o problemas de rendimiento pueden hacer que la aplicación deje de funcionar. Estos problemas afectan la confiabilidad de la aplicación.

5.5.3. Falta de mantenimiento o mejoras

Una aplicación que no recibe actualizaciones ni mejoras con el tiempo puede volverse obsoleta. Así que, es importante mantener el sistema actualizado para adaptarse a las necesidades de los usuarios.

5.5.4. Falta de valor para el usuario

Si los estudiantes no sienten que la aplicación les ayuda realmente a administrar su dinero, es posible que la abandonen. Por esta razón, nos debemos enfocar en resolver de forma efectiva el problema.

6. Tecnología

6.1. Lenguaje elegido

Vamos a usar el framework de Ionic, el cual está basado en HTML, CSS y TypeScript. Esto permite utilizar, si se desea, otros frameworks de JavaScript/TypeScript como Angular, React o Vue. Servirá para crear la aplicación como si fuera una página web, lo que facilitará el desarrollo de una forma más fácil comparado con otras alternativas de desarrollo móvil nativo. Además, Ionic permite crear aplicaciones multiplataforma, lo que significa que una misma base de código puede utilizarse para generar aplicaciones que funcionen tanto en dispositivos Android como en navegadores web. Otra ventaja de utilizar Ionic es que cuenta con una gran cantidad de componentes visuales ya preparados, como botones, menús, formularios y barras de navegación, lo que permite desarrollar interfaces de usuario de forma más rápida. También cuenta con una comunidad amplia y buena documentación, lo que facilita resolver problemas durante el desarrollo del proyecto.

6.2. Herramientas

Las herramientas que se utilizarán durante el desarrollo del proyecto son las siguientes:

- Ionic Framework
- Visual Studio Code
- GitHub

Ionic Framework será utilizado como la base principal para la creación de la aplicación. Visual Studio Code se utilizara como editor de código debido a su facilidad de uso, compatibilidad con múltiples lenguajes y extensiones que ayudan al desarrollo. GitHub permitirá almacenar el código del proyecto en línea y facilitar el trabajo colaborativo entre los integrantes del equipo.

6.3. Control de versiones (en GitHub)

Utilizaremos GitHub para poder controlar qué hacemos y qué cambios se van realizando en el proyecto. Cada modificación realizada en el código puede registrarse mediante commits, los cuales permiten dejar mensajes descriptivos como por ejemplo ”Fix de xxxx.o ”Función xxxx implementada”. Esto permite llevar un historial de cambios del proyecto y facilita regresar a versiones anteriores en caso de que algo deje de funcionar correctamente. Además, el control de versiones ayuda a que varias personas puedan trabajar en el mismo proyecto sin sobrescribir el trabajo de otros. La versión enviada se puede visualizar según la gráfica mostrada abajo, la cual representa el historial de cambios del repositorio.

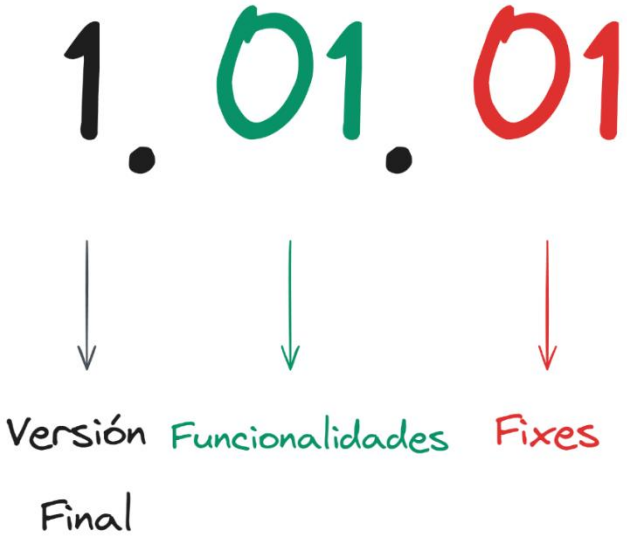

Figura 8. Descripción de versiones de Autoría propia

6.4. Herramientas de prueba

Esta parte puede actualizarse a futuro mientras vamos descubriendo cómo se utilizan herramientas específicas para pruebas automatizadas. Algunas opciones que podrían

utilizarse son Jasmine o Karma, las cuales permiten realizar pruebas de manera automática sobre el código desarrollado. Sin embargo, durante las primeras etapas del proyecto es probable que se realicen principalmente pruebas manuales. Estas pruebas pueden llevarse a cabo ejecutando la aplicación en navegadores o en dispositivos móviles reales. También se puede pedir apoyo a personas externas, como familiares o amigos, para probar la aplicación y obtener retroalimentación sobre su funcionamiento y facilidad de uso.

6.5. Infraestructura

La infraestructura utilizada para el desarrollo del proyecto incluye los siguientes elementos:

- El desarrollo se realizará en Visual Studio Code utilizando Node.js para gestionar dependencias y ejecutar el entorno de desarrollo.
- El código del proyecto se almacenará en repositorios de GitHub.
- El servidor de pruebas podrá ejecutarse localmente durante el desarrollo utilizando las herramientas de Ionic.
- El servidor final estará alojado en la nube para permitir el acceso a la aplicación desde diferentes dispositivos.
- Para la compilación de la aplicación en dispositivos Android se podrá utilizar Android Studio.

7. Referencias

[1] Salimi, S. (s.f.). Fail fast. Agile Academy. Disponible en: https://www. agile-academy.com/es/diccionario-agil/fallar-rapido/
[2] Villa, A. A. (2025, September 1). Qué es Smoke Test. Profile Software Services. Disponible en: https://profile.es/blog/que-es-smoke-test/

8. Anexos

8.1. Bitácora

Como tal no es la definición de una bitácora, solo será una sección en el documento donde semanalmente subiremos un reporte de nuestros avances a través de evidencias en jira, como esta sección tiene relación con la forma de evaluación y no directamente con el proyecto es posible que esta sección sea eliminada en la entrega final o se preserve para efectos de documentación.

Sprint 1.

- Semana 1 (01 - 08 de marzo 2026)
  En esta semana teníamos como meta las primeras tres secciones de la documentación, por lo que algunas secciones se dividieron en subactividades, pero finalmente

terminamos las 3 con éxito, luego de la revisión estuvimos de acuerdo con la entrega del documento y procedimos, a definir los planes de trabajo de la siguiente semana.

Actividad	Persona asignada	Prioridad	Estado	Start date	Fecha
~~SGGA-1~~ Definir el Proyecto	Sin asignar	Highest	FINALIZADA	02 mar 2026	08 mar 2026
~~SGGA-2~~ Creación de espacio en Jira	ANA PAULA SANTILL...	Medium	FINALIZADA	02 mar 2026	08 mar 2026
~~SGGA-4~~ Definición del Product Backlog	ANA PAULA SANTILL...	Medium	FINALIZADA	02 mar 2026	08 mar 2026
~~SGGA-18~~ Documentacion: Seccion 1	JARED GUSTAVO BEL...	Low	FINALIZADA	02 mar 2026	08 mar 2026
~~SGGA-19~~ Documentacion:Seccion 2	luis Eduardo Sariñana...	Low	FINALIZADA	02 mar 2026	08 mar 2026
~~SGGA-20~~ Documentacion:Seccion 3	ANA PAULA SANTILL...	Low	FINALIZADA	02 mar 2026	08 mar 2026
~~SGGA-31~~ Planeacion del Proyecto	ANA PAULA SANTILL...	Medium	FINALIZADA	01 mar 2026	08 mar 2026
~~SGGA-5~~ Análisis de requerimientos del sistema	JARED GUSTAVO BEL...	Highest	EN CURSO	08 mar 2026	15 mar 2026

Figura 9. Fragmento Backlog semana del 01 al 08 de marzo de Autoría propia

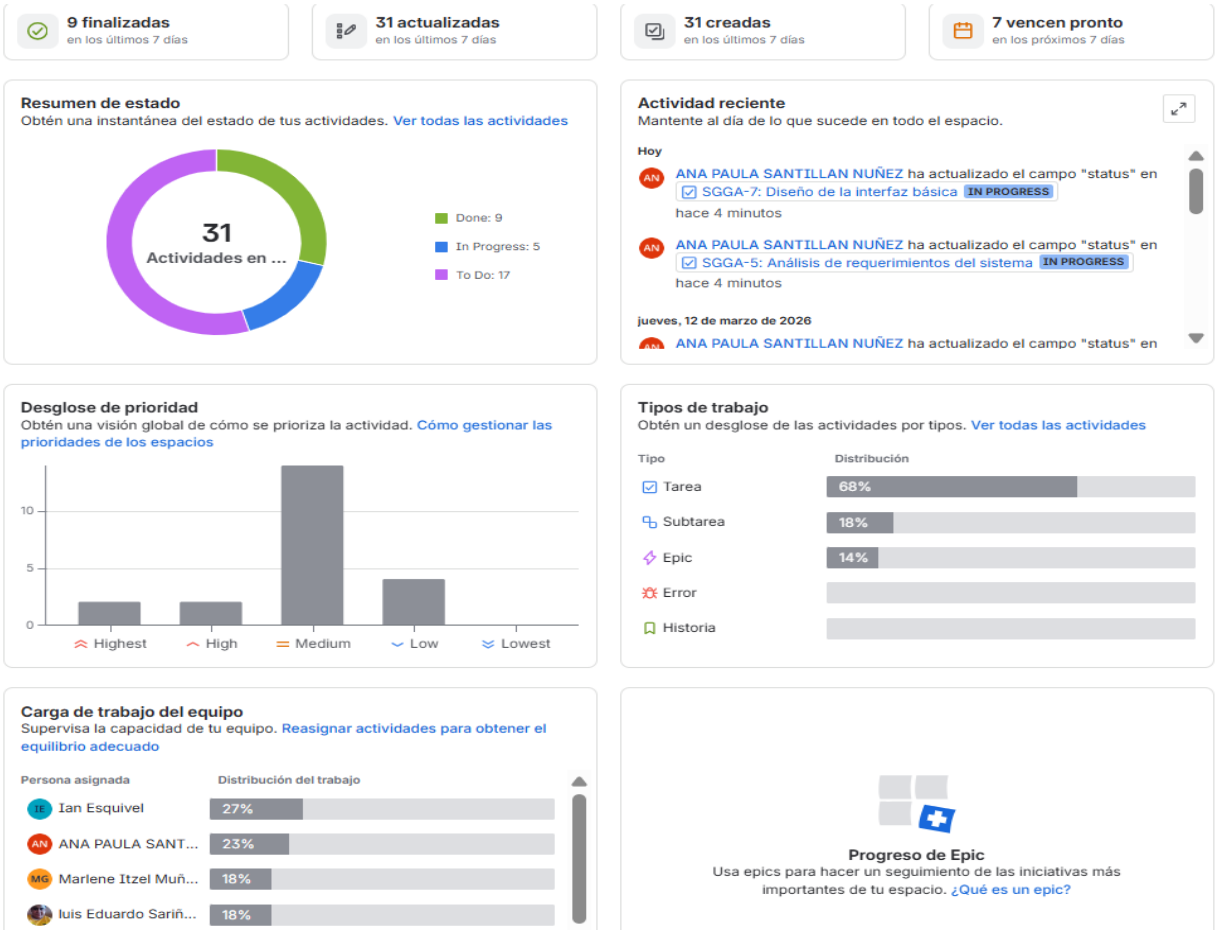

Figura 10. Fragmento del dashboard de Jira de Autoría propia.

- Semana 2 (09 - 15 de marzo 2026)

Nuevamente esta semana teníamos como meta las otras tres secciones de la documentación, por lo que algunas secciones se dividieron en subactividades, pero finalmente terminamos las 3 con éxito, luego de la revisión estuvimos de acuerdo con la entrega del documento y procedimos, a definir los planes de trabajo de la siguiente semana, además realizamos otras decisiones en conjunto y discutimos sobre la estructura del proyecto

pues teníamos que definir algunas parte como equipo para que todos estuviéramos acorde, por tanto hicimos un pequeño diseño de la aplicación.

Figura 11. Diseño informal del proyecto de Autoría propia

En las siguientes figuras se observa el avance final de esta semana, hicimos una pausa en las actividades SGGA-6 y SGGA-7 porque recibimos la observación por parte de nuestro docente, aclarando que en esta etapa no era necesaria la codificación del proyecto.

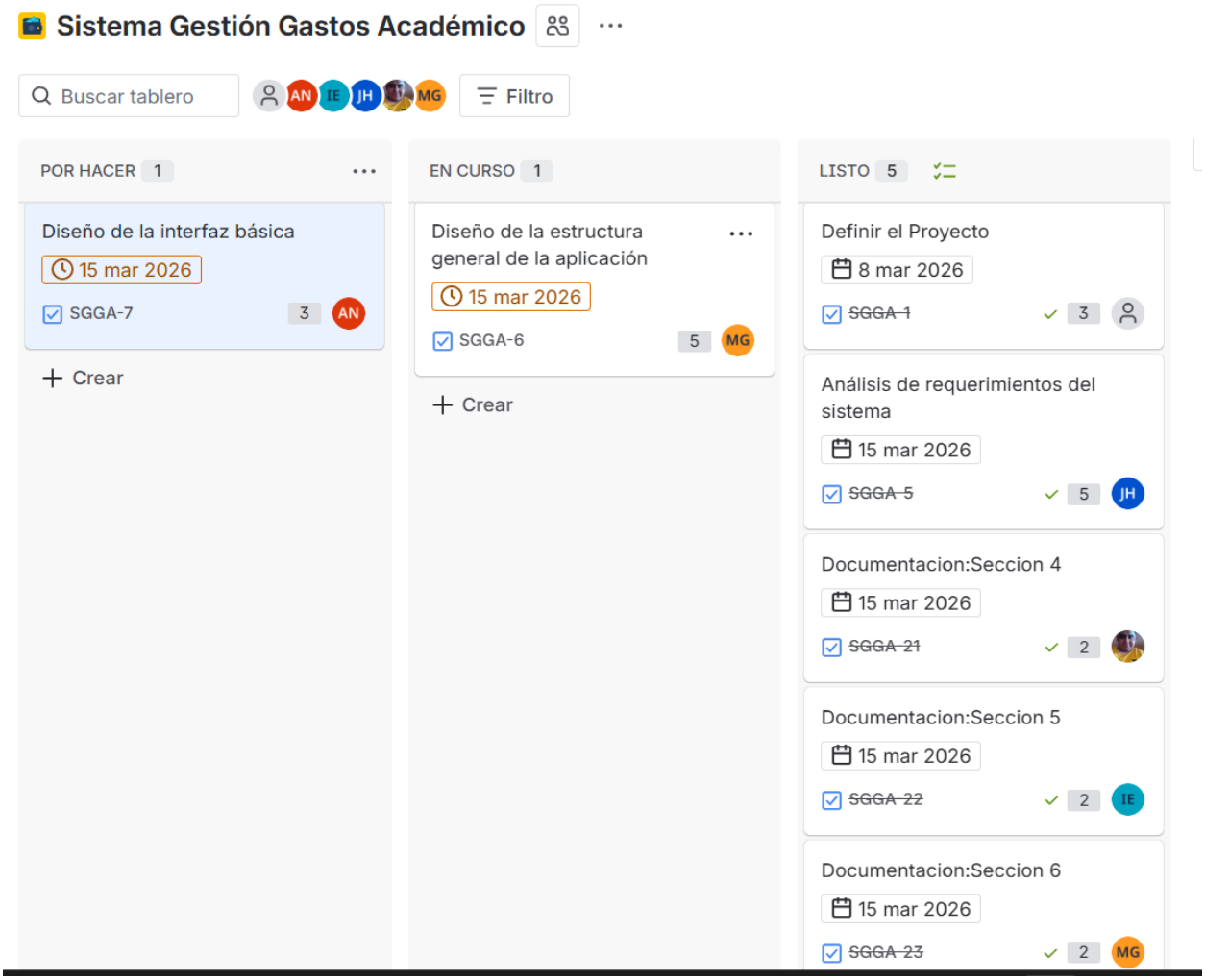

Figura 12. Kanban del proyecto semana 08 al 15 de marzo de Autoría propia

A continuación mostramos el estatus del sprint 1 al llegar a su fin.

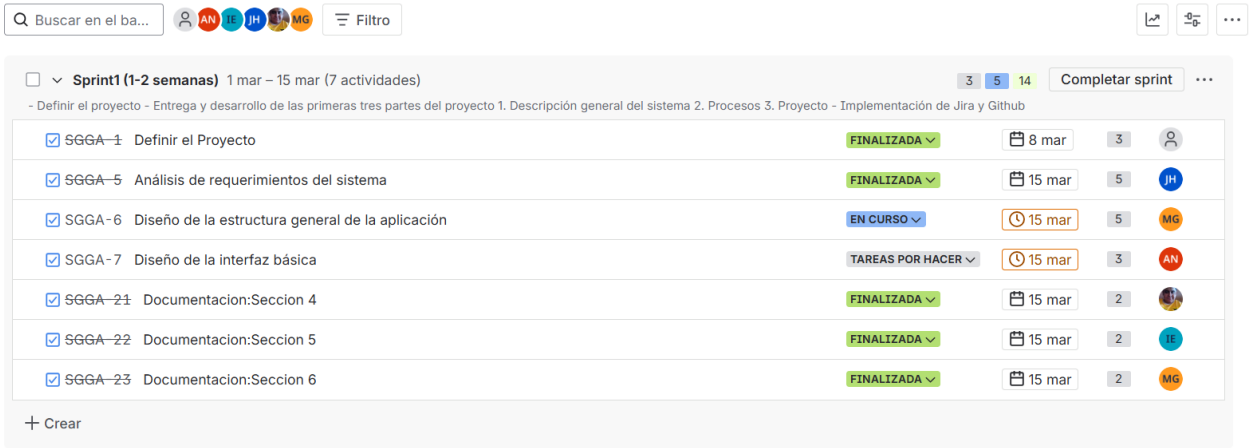

Figura 13. Sprint 1 semana 08 al 15 de marzo de Autoría propia

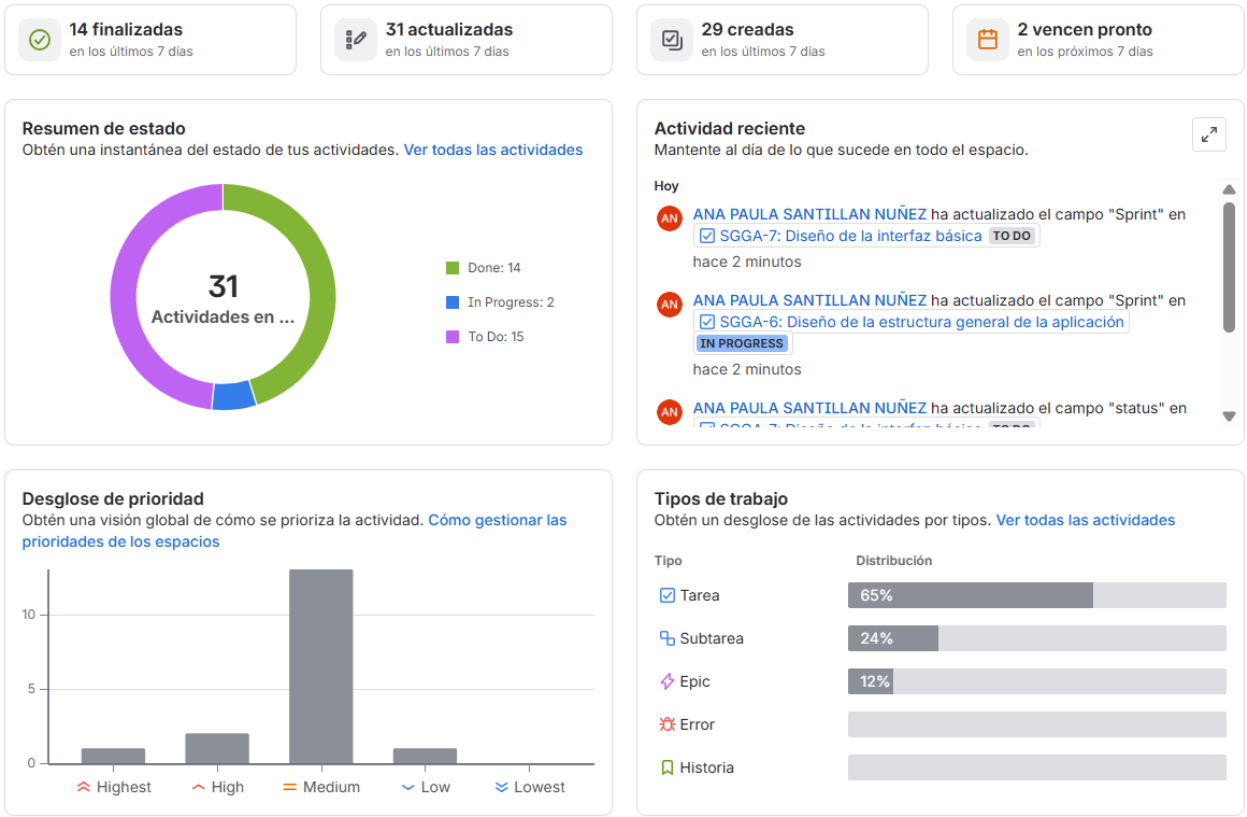

Figura 14. Dashboard de la semana 08 al 15 de marzo de Autoría propia

8.2. Repositorio de Github

Usamos GitHub en este proyecto porque nos permite gestionar y colaborar en el código de manera organizada y segura. A través de su sistema de control de versiones basado en Git, podemos registrar todos los cambios realizados en el proyecto, facilitar que varios desarrolladores trabajen al mismo tiempo sin perder historial y revisar modificaciones antes de integrarlas al código principal. Además, GitHub ofrece herramientas para seguimiento de tareas, revisión de código y automatización de procesos, lo que mejora la calidad del software y hace más eficiente el trabajo en equipo durante el desarrollo del proyecto.

[Repositorio de Github Sistema Control Gastos (bajo el modelo 4P+T)](https://github.com/AnaSantillanal246102/Sistema_Control_Gastos_4PT)

Link del Repositorio: https://github.com/AnaSantillanal246102/Sistema_Control_Gastos_4PT

9. Tablas

9.1. Tabla de contenido

1. Descripción general del sistema. 2
   1.1. Nombre..... 2
   1.2. Problema que resuelve..... 2
   1.3. Usuarios principales 2
   1.4. Objetivo general 3
   1.5 Objetivos Específicos 3
2. Procesos (de la aplicación) 3
   2.1 Perfil de usuarios..... 3
   2.2 Casos de Uso Principales..... 4
   2.3 Descripción de los Procesos Principales 4
      2.3.1. Proceso 1: Registro y categorización Inteligente 4
      2.3.2. Proceso 2: Monitoreo de Alertas de Solvencia 4
   2.4. Proceso de desarrollo..... 5
      2.4.1. Metodología de desarrollo elegida 5
      2.4.2. Justificación 5
   2.5. Actividades principales 6
      2.5.1. Análisis / Product Backlog..... 6
      2.5.2. Diseño / Planificación..... 6
      2.5.3. Desarrollo..... 7
      2.5.4.Pruebas 7
      2.5.5. Entrega / Review / Retrospective 7
   2.6. Flujo de trabajo..... 8
   2.7. Control de calidad..... 9
      2.7.1. ¿Cómo evitarán errores? 9
      2.7.2. ¿Qué prácticas utilizaran?..... 9
3. Proyecto 9
   3.1. Definición del proyecto 9
      3.1.1. Alcance 10

3.1.2. Entregables 10
3.2. Planificación básica (en Jira) 11
3.2.1. Cronograma 11
3.2.2. Lista de Actividades - Backlog 12
3.2.3. Sprints 14
3.3. Recursos 16
3.3.1. Humanos 16
3.3.2. Técnicos 16
3.4. Riesgos identificados 16
3.5. Métricas del proyecto 17
3.5.1. Tiempo 17
3.5.2. Costo estimado 17
3.5.3. Indicadores del avance 17
4. Personas 17
4.1. Roles 17
4.1.1. Product Owner 17
4.1.2. Scrum Master 17
4.1.3. Development Team 18
4.1.4. Resumen de roles 18
4.2. Responsabilidades 19
4.3. Estrategia de comunicación 19
4.4. Reglas internas del equipo 20
4.5. Reglas para el manejo de conflictos 20
5. Producto 20
5.1. Funcionalidades principales 20
5.2. Requisitos Básicos 21
5.2.1. Requisitos Funcionales (RF) 21
5.2.2. Requisitos No Funcionales (RNF) 21
5.3. Características del software 22
5.3.1. Usabilidad 22
5.3.2. Rendimiento y rapidez 22
5.3.3. Seguridad 22
5.3.4. Confiabilidad 23

5.3.5. Disponibilidad 23
5.3.6. Escalabilidad 23
5.4. Criterios de calidad 23
5.4.1. Correcto funcionamiento de las funcionalidades 23
5.4.2. Precisión de los cálculos 24
5.4.3. Experiencia de usuario 24
5.4.4. Mantenibilidad del sistema 24
5.4.5. Pruebas del sistema 24
5.5. ¿Qué haría que el producto fracase? 24
5.5.1. Falta de facilidad de uso 25
5.5.2. Problemas técnicos o fallos del sistema 25
5.5.3. Falta de mantenimiento o mejoras 25
5.5.4. Falta de valor para el usuario 25
6. Tecnología 25
6.1. Lenguaje elegido 25
6.2. Herramientas 26
6.3. Control de versiones (en GitHub) 26
6.4. Herramientas de prueba 26
6.5. Infraestructura 27
7. Referencias 27
8. Anexos 27
8.1. Bitácora 27
8.2. Repositorio de Github 31
9. Tablas 32
9.1. Tabla de contenido 32
9.2. Índice de tablas 34
9.3. Tabla de figuras 35

9.2. Índice de tablas

Tabla 1. Registro detallado de los posibles riesgos del proyecto de Autoría Propia 16
Tabla 2. Resumen de roles del equipo conforme a la metodología de Autoría propia 18
Tabla 3. Descripción de responsabilidades de cada rol de Autoría propia 19

9.3. Tabla de figuras

Figura 1. Diagrama de metodología de Autoría propia 8
Figura 2. Cronograma de Proyecto Fragmento de planificación en Jira de Autoría propia 11
Figura 3. Fragmento del Backlog del proyecto en Jira de Autoría propia.....12
Figura 4. Fragmento de Backlog del proyecto en Jira de Autoría propia 13
Figura 5. Imagen Sprint 1 del Proyecto en Jira de Autoría propia.....14
Figura 6. Imagen Sprint 2 del Proyecto en Jira de Autoría propia.....14
Figura 7. Imagen Sprint 3 del Proyecto en Jira de Autoría propia.....15
Figura 8. Descripción de versiones de Autoría propia 26
Figura 9. Fragmento Backlog semana del 01 al 08 de marzo de Autoría propia.....28
Figura 10. Fragmento del dashboard de Jira de Autoría propia.....28
Figura 11. Diseño informal del proyecto de Autoría propia.....29
Figura 12. Kanban del proyecto semana 08 al 15 de marzo de Autoría propia 30
Figura 13. Sprint 1 semana 08 al 15 de marzo de Autoría propia.....30
Figura 14. Dashboard de la semana 08 al 15 de marzo de Autoría propia 31